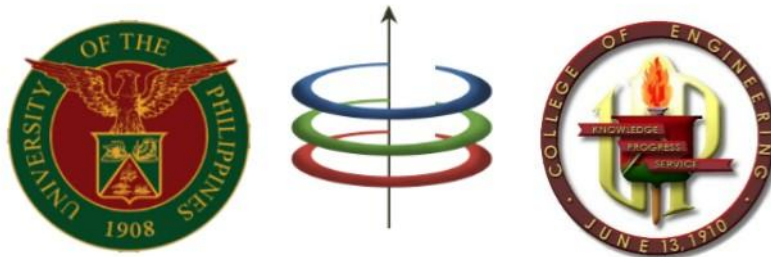


EEE 121 Lecture 10a: Problem Solving Approaches

Greedy Algorithms



The Greedy Approach

- ❑ A greedy algorithm has
 - a set of partial solutions from which the overall solution is built
 - an objective function we aim to minimize/maximize

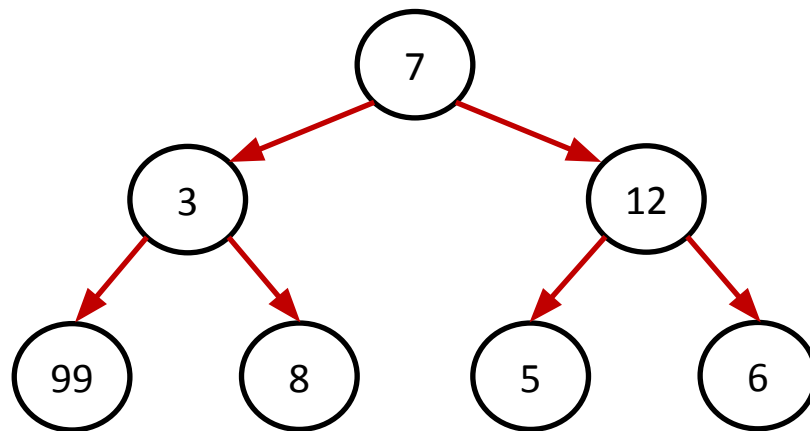
- ❑ Given a partial solution, consider all feasible extensions and choose the best extension (**greedy choice**)
 - **Advantages:** simple to design, often a “good” solution
 - **Disadvantages:** difficult to verify correctness or optimality

- ❑ **Caution:** In most cases, greedy approaches are suboptimal



Non-example for Greedy Approach

- ❑ **Problem:** Find the path from root to any leaf node such that sum of node values along the path is maximized
- ❑ **Greedy choice:** Select the larger children number at each step



Greedy solution: 7 $\square$ 12 $\square$ 6

Optimal solution: 7 $\square$ 3 $\square$ 99

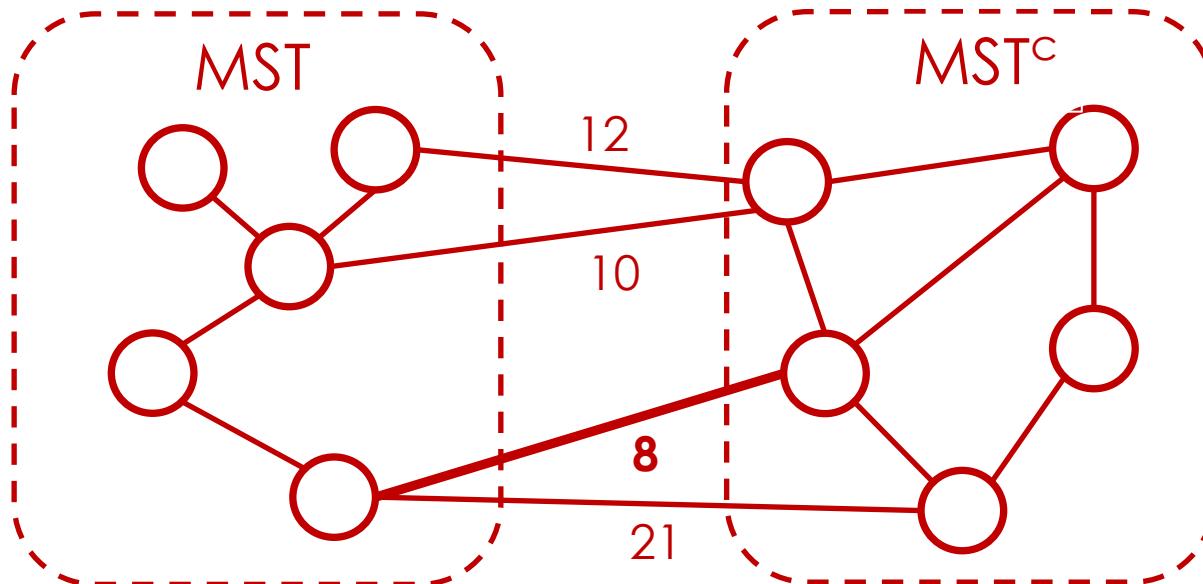
- ❑ Why greedy fails?

It makes decisions based only on the information it has at any one step, without regard to the overall problem



Example: Prim's Algorithm

- Prim's Algorithm is greedy
 - **Partial solution:** subgraph MST
 - **Feasible extensions:** add crossing edges to MST
 - **Greedy choice:** choose the minimum weight crossing edge



Example: Dijkstra's Algorithm

- Dijkstra's algorithm is greedy
 - **Partial solution:** subset of k vertices and known $d_s[v]$ to k vertices
 - **Feasible extensions:** process $(v, d_s[v])$ pair in the priority queue to update SPT
 - **Greedy choice:** Priority queue picks the smallest $d_s[v]$

Greedy choice

- pop the node x with min priority
- for each neighbor y of the popped node:
 - if $d_s[x] + w(x,y) < d_s[y]$:
 - $d_s[y] = d_s[x] + w(x,y)$
 - $pre[y] = x$
 - push y onto the priority queue with $d_s[y]$ as its priority

add feasible extension



Activity Selection Problem

- ❑ **Given:** list of activities with start time and finish times

$$(s_1, e_1), (s_2, e_2), \dots, (s_n, e_n)$$

- ❑ **Task:** Select the largest number of non-overlapping activities

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
e_i	4	5	6	7	8	9	10	11	12	13	13



Activity Selection Problem

- Possible “greedy” choices:
 - **be impulsive**: choose according to ascending start times
 - **Avoid commitment**: choose activities in ascending order of length.
 - **Finish fast**: choose activities in ascending order of end times.

Approach #1:

i	3	1	10	2	5	4	6	7	8	9	11
s _i	0	1	2	3	3	5	5	6	8	8	12
e _i	6	4	13	5	8	7	9	10	11	12	13



Activity Selection Problem

- Possible “greedy” choices:
 - **be impulsive**: choose according to ascending start times
 - **Avoid commitment**: choose activities in ascending order of length.
 - **Finish fast**: choose activities in ascending order of end times.

Approach #2:

i	11	2	4	1	8	6	9	7	5	3	10
s _i	12	3	5	1	8	5	8	6	3	0	2
e _i	13	5	7	4	11	9	12	10	8	6	13

Better than approach #1!



Activity Selection Problem

- ❑ Possible “greedy” choices:
 - **be impulsive**: choose according to ascending start times
 - **Avoid commitment**: choose activities in ascending order of length.
 - **Finish fast**: choose activities in ascending order of end times.

Approach #3:

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
e_i	4	5	6	7	8	9	10	11	12	13	13

Same as approach #2. Are they both optimal approaches?



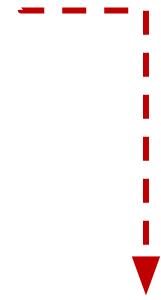
Activity Selection Problem

□ Let's consider this set of activities:

Approach #2

i	1	2	3	4	5	6	7	8
s_i	3	1	4	3	5	6	7	8
e_i	5	4	7	9	9	10	11	12

Approach #3



i	1	2	3	4	5	6	7	8
s_i	3	1	4	3	5	6	7	8
e_i	5	4	7	9	9	10	11	12

i	1	2	3	4	5	6	7	8
s_i	3	1	4	3	5	6	7	8
e_i	5	4	7	9	9	10	11	12

In some cases, approach 3 is better. Is it optimal in general?



Proving Greedy Algorithms

Pseudocode:

Runtime:
 $O(n \log(n))$
How?

```
def activity_selection(activities):  
    sort activities into ascending order by end time  
    S = {}  
    U = set of activities  
    while U not empty:  
        choose any activity with the earliest finishing time  
        add that activity to S  
        remove other activities that overlap with it from U  
    return S
```

- We need to prove two things about approach #3:
 1. **Feasibility** – The algorithm finds a feasible schedule of activities (i.e. it doesn't "schedule conflicting activities").
 2. **Optimality** – The algorithm finds an optimal schedule of activities (i.e. there isn't a better schedule available).



Feasibility of Approach #3

- ❑ **Loop Invariant:** Set S_i contains non-overlapping activities at iteration i
- ❑ **Proof:** The loop contains

while U **not** empty:

 choose any activity x with the earliest finishing time

 add that activity x to S

 remove other activities that overlap with x from U

- Before loop execution, set S_0 is empty set (Feasible!)
- At every iteration,
 - we add an activity from set U ($S_i \sqsubset S_{i+1}$)
 - After removal of activities, set U only contains activities that don't conflict with activities selected from set S (feasibility is maintained at every iteration!)



Optimality of Approach #3

- Let $f(i, \mathbf{S})$ denote the time that the i -th activity in $\mathbf{S}$ finishes in schedule $\mathbf{S}$ (sorted in ascending finish time).

$$f(i, \mathbf{S}) = e_i \text{ such that } (s_i, e_i) \in \mathbf{S}$$

- **Intuition:** Consider an arbitrary “optimal” schedule $\mathbf{S}^*$, then show that our greedy algorithm produces a schedule $\mathbf{S}$ no worse than $\mathbf{S}^*$.
- We will prove the following Lemma:

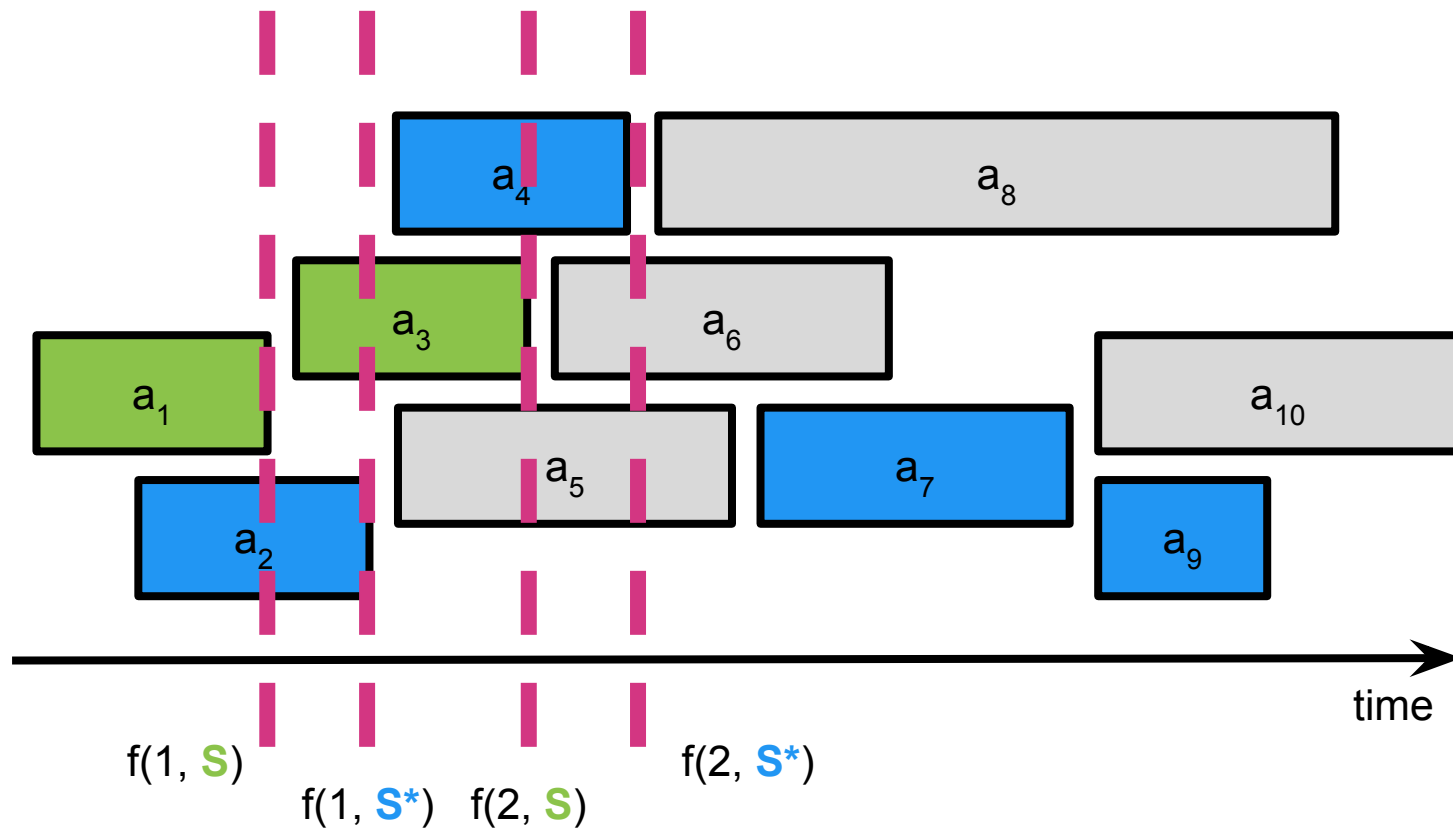
Lemma: For any $1 \leq i \leq |\mathbf{S}|$, we have $f(i, \mathbf{S}) \leq f(i, \mathbf{S}^*)$.

After scheduling i activities according to the greedy algorithm, you will be at most as late as if you scheduled i activities according to an optimal solution.



Optimality of Approach #3

For any $1 \leq i \leq |S|$, we have $f(i, \mathbf{s}) \leq f(i, \mathbf{s}^*)$.



Optimality of Approach #3

 **Lemma:** For any $1 \leq i \leq |\mathbf{S}|$, we have $f(i, \mathbf{S}) \leq f(i, \mathbf{S}^*)$.

 Proof of Lemma:

- **Base Case:** the first activity the greedy algorithm selects must be an activity that ends no later than any other activity, so $f(1, \mathbf{S}) \leq f(1, \mathbf{S}^*)$.

 $\mathbf{U}$ is sorted according to ascending finish time

- **Inductive Step:** Suppose $f(i, \mathbf{S}) \leq f(i, \mathbf{S}^*)$ is true.
 - $f(i, \mathbf{S}) \leq f(i, \mathbf{S}^*) \Rightarrow$ the i -th activity in $\mathbf{S}$ finishes before the i -th activity in $\mathbf{S}^*$
 - Since $(i+1)$ -th activity in $\mathbf{S}^*$ starts after i -th activity in $\mathbf{S}^*$ ends, it must also start after i -th activity in $\mathbf{S}$ end. (transitivity)
 - $(i+1)$ -th activity in $\mathbf{S}^*$ must be in $\mathbf{U}$ when the greedy algorithm selects the activity in $\mathbf{U}$ with the lowest end time, we have $f(i+1, \mathbf{S}) \leq f(i+1, \mathbf{S}^*)$,



Optimality of Approach #3

- ❏ **Claim:** Let S^* be the “optimal” solution and S be the result from the approach #3. Then $|S| \geq |S^*|$
- ❏ We will proceed by **proof by contradiction**:
 - Suppose that $|S| < |S^*|$ and let $k = |S|$
 - From the lemma, we know $f(k, S) \leq f(k, S^*)$
 - $|S| < |S^*| \Rightarrow$ there is $(k+1)$ -th activity in S^* , such s_{k+1} is $\geq f(k, S^*)$ and $\geq f(k, S)$
 - after the greedy algorithm added its k -th activity to S , the $(k+1)$ -th activity from S^* would still belong to U
 - greedy algorithm ended after k activities, so U must have been empty

Contradiction!



”Prioritized” Scheduling Problem

□ Given:

- List of all the task you need to complete ($i = 1, \dots, n$)
- Time required to complete each task ($T[1], \dots, T[n]$)
- Priority (or weight) of each work ($P[1], \dots, P[n]$)

□ Suppose the tasks are accomplished by order

$$a_1, a_2, a_3, \dots, a_n$$

□ Completion time of task j-th task is

$$C(a_j) = T[a_1] + T[a_2] + \dots + T[a_j]$$

□ jobs that

- are queued up at the beginning have a shorter completion times
- are queued up towards the end have longer completion times
- have high priority is preferred to be completed sooner



”Prioritized” Scheduling Problem

- Consider the following objective function is a weighted sum of completion times:

$$F = P[a_1] * C(a_1) + P[a_2] * C(a_2) + + P[a_n] * C(a_n)$$

- **Goal:** Minimize **F** by reordering the task
- Special cases to consider:
 1. If the time required to complete different tasks is the same i.e. **$T[i] = T[j]$ where $1 \leq i, j \leq N$** , but they have different priorities. (optimal order?)
 2. If the priorities of different tasks are the same i.e. **$P[i] = P[j]$ where $1 \leq i, j \leq N$** but they have different lengths. (optimal order?)



”Prioritized” Scheduling Problem

Case 1

- ❑ Consider the objective function that you need to minimize. Assume that the time required to complete the different tasks is t .
- ❑ $T[i] = t$ where $1 \leq i \leq N$
- ❑ Irrespective of what sequence is used, the completion time for each task will be as follows:
$$C(1) = T[1] = t$$
$$C(2) = T[1] + T[2] = 2 * t$$
$$C(3) = T[1] + T[2] + T[3] = 3 * t$$
$$\dots$$
$$C(n) = n * t$$
- ❑ To make the objective function as small as possible the highest priority must be associated with the shortest completion time.



”Prioritized” Scheduling Problem

Case 2

- ❑ If the priorities of different tasks are the same, favor the task that requires the least amount of time to complete.

- ❑ Assume that the priorities of the different tasks is **p**.

$$F = P[1] * C(1) + P[2] * C(2) + + P[N] * C(N)$$

$$F = p * C(1) + p * C(2) + + p * C(N)$$

$$F = p * (C(1) + C(2) + + C(N))$$

- ❑ To minimize the value of F, you must minimize (C(1) + C(2) + + C(N))
- ❑ can be done if you start working on the tasks that require the shortest time to complete.



”Prioritized” Scheduling Problem

- ❑ **Intuition:** Give preference to tasks that
 - have a higher priority
 - take less time to complete
- ❑ It's hard to optimize since we are considering two parameters in our objective function
- ❑ Can we combine them into a single “score”?
- ❑ Two possible approaches:
 - **Approach #1:** order the jobs by **decreasing value of ($P[i] - T[i]$)**
 - **Approach #2:** order the jobs by **decreasing value of ($P[i] / T[i]$)**
- ❑ One of them is correct. Which one?



Approach #1 is incorrect

Proof by counterexample:

- Consider this two activity example:

$$T = \{5, 2\} \text{ and } P = \{3, 1\}$$

- According to the approach #1 $(P[1] - T[1]) < (P[2] - T[2])$, therefore, the second task should be completed first and your objective function will be:

$$F = P[2] * C(2) + P[1] * C(1) = 1 * 2 + 3 * 7 = 23$$

- According to approach #2 $(P[1] / T[1]) > (P[2] / T[2])$, therefore, the first task should be completed first and your objective function will be:

$$F = P[1] * C(1) + P[2] * C(2) = 3 * 5 + 1 * 7 = 22$$



Optimality of Algorithm #2

- ❑ Simplifying assumptions:
 1. $P[i]/T[i]$ are different for all i
 2. $P[1]/T[1] > P[2]/T[2] > \dots > P[n]/T[n]$ (sorted according to $P[i]/T[i]$)

- ❑ Greedy solution is

$$\mathbf{A} = (1, 2, 3, \dots, n) \text{ (why?)}$$

- ❑ Proceed with **proof by contradiction**:
 - Assume an “optimal” solution is $\mathbf{B}$ not the same as $\mathbf{A}$. Then $\mathbf{B}$ must contain two consecutive jobs j and i such that $i < j$, i.e.

$$\mathbf{B} = (1, \dots, j, i, \dots, n)$$

- $\mathbf{B}$ can be obtained after swapping i and j .



Optimality of Algorithm #2

- Proceed with **proof by contradiction (continued)**:
 - No effect on completion time of tasks k before and after the pair (i, j)

1. If k is before (i, j): $C(k) = T[1] + \dots + T[k]$ (no terms involving i and j)

2. If k is after (i, j): $C(k) = T[1] + \dots T[i] + T[j] + \dots + T[k]$
 $= T[1] + \dots T[j] + T[i] + \dots + T[k]$

- Compute F_A and F_B

$$F_A = C(1)*P[1] + \dots + C(i) * P[i] + C(j) * P[j] + \dots + C(n)*P[n]$$

$$F_B = C(1)*P[1] + \dots + C(j) * P[j] + C(i) * P[i] + \dots + C(n)*P[n]$$

$C(i)$ and $C(j)$ are different in these two equations!



Optimality of Algorithm #2

□ Proceed with **proof by contradiction** (continued):

- Completion of task i and j in solution **A**

$$C_A(i) = T[1] + T[2] + \dots + T[i]$$

$$C_A(j) = T[1] + T[2] + \dots + T[i] + T[j]$$

- Completion of task i and j in solution **B**

$$C_B(i) = T[1] + T[2] + \dots + T[j] + T[i]$$

$$C_B(j) = T[1] + T[2] + \dots + T[j]$$

- $F_A - F_B = P[i]*(C_A(i) - C_B(i)) + P[j]*(C_A(j) - C_B(j))$
 $= P[i]*(-T[j]) + P[j]*(T[i]) < 0$ (why?)

- Since $F_A < F_B$, F_B is not optimal (contradiction)



Fractional Knapsack

- ❑ **Given:** (value, weight) of N items
- ❑ **Goal:** Put these items in a knapsack of capacity W to get the maximum total profit in the knapsack.

Item	1	2	3	$W = 50$
Value	60	100	120	
Weight	10	20	30	

“Fractional” means we can break items for maximizing the total value of the knapsack

Enables us to be “greedy”



Fractional Knapsack

□ Greedy Solution:

Greedy choice: item with current best (profit/weight)

Calculate (profit/weight) for each item.

Sort items in decreasing order of their ratios.

res = 0, curr_cap = given_cap.

do the following for every item *i* in the sorted order:

 if(weight[i] ≤ curr_cap)

 res = res + val[i]

 curr_cap = curr_cap - weight[i]

 else

 res = res + (val[i]/weight[i])*curr_cap

 curr_cap = 0;

return res.

Try to prove this on your own (HINT: do a proof by contradiction)



When to be Greedy?

- ❑ A problem must comprise these two components for a greedy algorithm to work:
 1. It has **optimal substructures**. The optimal solution for the problem contains optimal solutions to the sub-problems.
 2. It has a **greedy property** if you make a choice that seems the best at the moment and solve the remaining sub-problems later, you still reach an optimal solution.
- ❑ hard to prove the **greedy property**!
 - usual approach to prove is via proof by contradiction
 - disprove via finding counterexamples (also hard)

